1. Symmetric Key Encryption: Secure File Transfer System

Problem Statement

Design and implement a secure file transfer system using AES-256 encryption in CBC mode. The system should consist of:

- A sender application that encrypts files
- A receiver application that decrypts files
- A secure key exchange mechanism using RSA or ECDH
- Authentication to prevent unauthorized decryption
- Integrity verification using HMAC

Implementation Requirements

1. **Key Management:**
    - Generate 256-bit AES key using cryptographically secure random generator
    - Implement RSA-2048 for secure key exchange
    - Store keys in encrypted key vault with password protection
2. **Encryption Process:**

```
def encrypt_file(input_file, output_file, key):
    # Generate random IV
    # Use AES-256-CBC mode
    # Add HMAC-SHA256 for integrity
    # Output: IV + ciphertext + HMAC
```

3. **Decryption Process:**

```
def decrypt_file(input_file, output_file, key):
    # Verify HMAC first
    # Then decrypt with AES-256-CBC
    # Return error if HMAC fails
```

4. **Secure Key Exchange:**
    - Use hybrid encryption: RSA encrypts AES key
    - Implement key derivation with PBKDF2

Expected Output

=== SENDER SIDE ===

File to encrypt: document.pdf

Generating AES-256 key...

Encrypting file...

IV generated: 89a7f3d2e1c4b5a6...

HMAC computed: sha256-89f3a7...

Encrypted file saved as: document.pdf.enc

AES key encrypted with RSA: rsa_encrypted_key.bin

=== KEY EXCHANGE ===

Public key sent to receiver: receiver_pub.pem

Encrypted AES key sent securely

=== RECEIVER SIDE ===

Received encrypted key: rsa_encrypted_key.bin

Decrypting AES key with private key...

AES key recovered successfully

Loading encrypted file: document.pdf.enc

Verifying HMAC: ✓ Valid

Decrypting file...

Original file restored: document_decrypted.pdf

SHA256 match with original: ✓ Verified

---

2. Asymmetric Key Encryption: Secure Messaging App

Problem Statement

Develop an end-to-end encrypted messaging application using RSA-2048/OAEP. The system should include:

- Key pair generation and management
- Secure message encryption/decryption
- Digital signatures for authentication
- Message integrity and non-repudiation

Implementation Requirements

1. **Key Generation Module:**

```
def generate_key_pair(user_id):
    # Generate RSA-2048 keys
    # Save public key to keys/[user_id]_pub.pem
    # Save encrypted private key to keys/[user_id]_priv.pem
    # Return key fingerprints
```

2. **Message Encryption:**

```
def encrypt_message(message, recipient_public_key):
    # Generate random session key (AES-128)
    # Encrypt message with session key
    # Encrypt session key with RSA-OAEP
    # Add digital signature
    # Output: JSON with encrypted components
```

3. **Message Decryption:**

```
def decrypt_message(encrypted_package, private_key):
    # Verify signature
    # Decrypt session key
    # Decrypt message
```

 # Return plaintext or error

Expected Output

=== USER REGISTRATION ===

User: alice

Generating RSA-2048 key pair...

Public key fingerprint: SHA256:4a8b9c...

Private key encrypted with password

Keys saved to: keys/alice_priv.pem, keys/alice_pub.pem

=== SENDING MESSAGE ===

Sender: alice

Recipient: bob

Message: "Meeting at 3 PM tomorrow"

Encrypting with bob's public key...

Session key: aes128-7d9e2f...

Encrypted package:

{
  "ciphertext": "base64_encrypted_data",
  "encrypted_key": "rsa_oaep_encrypted",
  "signature": "sha256_with_rsa",
  "sender": "alice",
  "timestamp": "2025-02-03T14:30:00Z"
}

=== RECEIVING MESSAGE ===

User: bob

Loading encrypted message...

Verifying alice's signature: ✓ Valid

Decrypting session key...

Decrypting message...

Plaintext: "Meeting at 3 PM tomorrow"

Message integrity: ✓ Verified

Sender authentication: ✓ Confirmed

---

3. Password Validator System

Problem Statement

Create a comprehensive password validation system that:

- Enforces strong password policies
- Checks against known password breaches
- Implements progressive strength scoring
- Provides user-friendly feedback

Implementation Requirements

1. **Policy Enforcement:**

```
def validate_password(password):
    # Minimum 12 characters
    # Mixed case, numbers, special chars
    # No common patterns (qwerty, 12345)
    # Not similar to username/email
    # Not in breached passwords database
```

2. **Strength Scoring:**

```
def password_strength(password):
    # Entropy calculation
    # Common pattern detection
    # Dictionary attack resistance
    # Score: 0-100 with feedback
```

3. **Breach Check:**

```
def check_breach(password_hash_prefix):
    # Use HaveIBeenPwned API
    # Check against known breaches
    # Local bloom filter for caching
```

Expected Output

=== PASSWORD VALIDATION ===

Enter new password: **********

VALIDATION RESULTS:

✓ Length: 14 characters (minimum 12)

✓ Contains uppercase letters: 3

✓ Contains lowercase letters: 6

✓ Contains numbers: 2

✓ Contains special characters: 3

✗ Contains common pattern: "qwerty" detected

✗ Similar to username "johnsmith": 65% match

✓ Not in known data breaches

STRENGTH ANALYSIS:

Entropy: 78 bits

Estimated crack time: 45 years

Strength score: 75/100

RECOMMENDATIONS:

1. Avoid keyboard patterns like "qwerty"

2. Make less similar to personal information

3. Consider adding more random characters

FINAL STATUS: Password acceptable but could be stronger

---

4. Hash Function: Secure Password Storage

Problem Statement

Implement a secure password storage system using SHA-256 with salting and key stretching. The system should:

- Register users with secure password hashing
- Authenticate users securely
- Prevent rainbow table attacks
- Implement rate limiting

Implementation Requirements

1. **Registration Module:**

```
def register_user(username, password):
    # Generate random 32-byte salt
    # Use PBKDF2-HMAC-SHA256 with 100,000 iterations
    # Store: username, salt, hash, iteration count
```

2. **Authentication Module:**

```
def authenticate(username, password):
    # Retrieve salt and stored hash
    # Recompute hash with same parameters
    # Constant-time comparison
    # Implement exponential backoff on failures
```

3. **Database Schema:**

```
sql
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE,
    salt BINARY(32),
    password_hash BINARY(32),
    iterations INT DEFAULT 100000,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    failed_attempts INT DEFAULT 0,
    locked_until TIMESTAMP NULL
);
```

Expected Output

=== USER REGISTRATION ===

Username: alice

Password: **********

Generating salt: 7a4b9c2d8e1f3a5b...

Applying PBKDF2-HMAC-SHA256 (100,000 iterations)...

Password hash: e3b0c44298fc1c14...

Storing in database...

REGISTRATION SUCCESSFUL:

Username: alice

Salt stored: [32 bytes]

Hash stored: [32 bytes]

Iterations: 100000

=== USER LOGIN ===

Username: alice

Password: **********

Retrieving salt and hash...

Recomputing hash...

Hash comparison: ✓ Match

Login successful!

=== FAILED LOGIN ===

Username: alice

Password: wrongpassword

Hash comparison: ✗ No match

Failed attempts: 1/5

Warning: 4 attempts remaining before lockout

5. User Authentication and Access Control Portal

Problem Statement

Create a web-based role-based access control system with MySQL backend that manages student information with three user roles: Admin, Editor, Viewer.

Implementation Requirements

1. **Database Schema:**

```
sql

-- Users table

CREATE TABLE users (

  user_id INT PRIMARY KEY AUTO_INCREMENT,

  username VARCHAR(50) UNIQUE,

  password_hash VARCHAR(255),

  role ENUM('admin', 'editor', 'viewer'),

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);


-- Student information

CREATE TABLE stud_info (

  roll VARCHAR(20) PRIMARY KEY,

  name VARCHAR(100),

  age INT,

  branch VARCHAR(50),

  hometown VARCHAR(100),

  created_by INT,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (created_by) REFERENCES users(user_id)

);


-- Audit log

CREATE TABLE audit_log (

  log_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    user_id INT,
    action VARCHAR(50),
    table_name VARCHAR(50),
    record_id VARCHAR(20),
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ip_address VARCHAR(45)
);
```

2. **Role-based Permissions:**
    - **Admin:** CRUD all records, manage users, view audit logs
    - **Editor:** Create, Read, Update records (no delete)
    - **Viewer:** Read-only access
3. **Web Interface Components:**
    - Login page with CSRF protection
    - Dashboard based on role
    - Student management forms
    - Audit log viewer (admin only)

Expected Output

=== LOGIN PORTAL ===

URL: https://rbac-portal.local/login

Username: admin

Password: **********

Authentication successful!

Role: Administrator

Session token generated (HTTP-only, Secure)

=== ADMIN DASHBOARD ===

Welcome, Administrator!

Menu:

1. Student Management - All operations

2. User Management - Add/Edit/Delete users

3. Audit Logs - View all activities

4. System Settings

Student List:

```
+--------+----------------+-----+-----------+-------------+
| Roll   | Name           | Age | Branch    | Hometown    |
+--------+----------------+-----+-----------+-------------+
| CS101  | John Smith     | 20  | CSE       | New York    | [Edit] [Delete]
| EC102  | Sarah Johnson  | 21  | ECE       | Chicago     | [Edit] [Delete]
+--------+----------------+-----+-----------+-------------+
```

=== EDITOR DASHBOARD ===

Welcome, Editor!

Menu:

1. Student Management - Add/Edit (no delete)

2. View Students

Permissions:

✓ Add new students

✓ Edit existing students

✗ Delete students

✗ Manage users

✗ View audit logs

=== VIEWER DASHBOARD ===

Welcome, Viewer!

Menu:

1. View Students (Read-only)

Student List (Read-only):

```
+--------+----------------+-----+-----------+
| Roll   | Name           | Age | Branch    |
+--------+----------------+-----+-----------+
| CS101  | John Smith     | 20  | CSE       |
| EC102  | Sarah Johnson  | 21  | ECE       |
+--------+----------------+-----+-----------+
```

=== AUDIT LOG (Admin Only) ===

```
+---------------------+---------+----------------+----------+---------------+
| Timestamp           | User    | Action         | Record   | IP Address    |
+---------------------+---------+----------------+----------+---------------+
| 2025-02-03 14:30:22 | admin   | CREATE         | CS103    | 192.168.1.100 |
```

| 2025-02-03 14:28:15 | editor1 | UPDATE | EC102 | 192.168.1.101 |

| 2025-02-03 14:25:47 | viewer1 | SELECT | * | 192.168.1.102 |

+---------------------+---------+----------------+----------+---------------+

---

6. Symmetric Searchable Encryption

Problem Statement

Build an encrypted database that supports keyword search without decrypting the entire dataset using symmetric cryptography.

Implementation Requirements

1. SSE Scheme (Deterministic Encryption):
    - Each document encrypted separately with AES
    - Build encrypted index from keywords
    - Search token generated from keyword
    - Match against encrypted index
2. Index Structure:

```
class EncryptedIndex:
  def __init__(self):
    self.index = {}  # word -> [encrypted_doc_ids]
    self.documents = {}  # encrypted_doc_id -> encrypted_content

  def add_document(self, doc_id, text, encryption_key):
    # Extract keywords
    # Encrypt each keyword: E(K, word)
    # Encrypt document: E(K, text)
    # Update index

  def search(self, keyword, encryption_key):
    # Generate search token: E(K, keyword)
    # Lookup in index
    # Return matching encrypted documents
```

Expected Output

=== SYMMETRIC SEARCHABLE ENCRYPTION ===

Database: Encrypted Medical Records

Documents: 10,000 patient records

Encryption: AES-256-CTR

Index type: Inverted index with deterministic encryption

=== DATABASE SETUP ===

Encryption key: K (256-bit)

Building encrypted index...

Processing documents:

[1/10000] Document: patient_0001.txt

Keywords: ["diabetes", "insulin", "2025-01-15", "dr_smith"]

Encrypting keywords:

E(K, "diabetes"): enc_d4b2e8a7c6...

E(K, "insulin"): enc_9a7b5c3d2e...

E(K, "2025-01-15"): enc_3c8d7e9f1a...

E(K, "dr_smith"): enc_5b9a8c7d6e...

Encrypting document content: enc_7d2e4f6a8b...

Updating index...

Index statistics:

Total keywords: 45,327 unique terms

Index size: 42MB (compressed)

Document storage: 850MB encrypted

Setup time: 12 minutes 45 seconds

=== SEARCH OPERATION ===

Search query: "diabetes AND insulin"

Step 1: Token generation

Keyword1: "diabetes" -> E(K, "diabetes") = enc_d4b2e8a7c6...

Keyword2: "insulin" -> E(K, "insulin") = enc_9a7b5c3d2e...

Tokens sent to server: [enc_d4b2e8a7c6..., enc_9a7b5c3d2e...]

Step 2: Server-side search

Lookup enc_d4b2e8a7c6... in index

Matches: [enc_doc_742, enc_doc_1589, enc_doc_2345, ...]

Count: 1,247 documents

Lookup enc_9a7b5c3d2e... in index

Matches: [enc_doc_742, enc_doc_4567, enc_doc_8910, ...]

Count: 893 documents

Intersection (AND): 327 documents

Search time: 8ms

Step 3: Result retrieval

Returning 327 encrypted documents

Total size: 27.8MB encrypted

Transfer time: 1.2 seconds

Step 4: Client-side decryption

Decrypting 327 documents with key K

Plaintext recovery...

Search complete: 327 relevant documents found

=== PERFORMANCE ANALYSIS ===

Search speed:

Index lookup: 0.2ms per keyword

Set operations: 0.5ms per operation

Total search: 1-10ms typical

Storage overhead:

Index size: 5-10% of original data

Encryption overhead: 16 bytes per AES block

Security properties:

✓ Data confidentiality maintained

✓ Search pattern hidden (deterministic encryption leaks)

✓ Forward secrecy: ✗ (new documents reveal new keywords)

✓ Backward secrecy: ✗ (requires re-encryption)

=== ADVANCED SEARCH FEATURES ===

Boolean queries supported:

✓ AND: diabetes AND insulin

✓ OR: diabetes OR hypertension

✓ NOT: diabetes NOT type1

Phrase search:

"type 2 diabetes" as separate tokens

Client-side post-filtering

Range queries (dates):

Encrypt each date separately

Client requests range, server returns superset

Client filters results

7. Database Access Control Mechanism

Problem Statement

Implement a fine-grained database access control system with SQL injection prevention and role-based permissions for CRUD operations.

Implementation Requirements

1. **Security Layers:**
    - Input validation and sanitization
    - Prepared statements only
    - Role-based query filtering
    - Query logging and monitoring
2. **Access Control Matrix:**

```
PERMISSIONS = {
  'admin': {
    'stud_info': ['SELECT', 'INSERT', 'UPDATE', 'DELETE'],
    'users': ['SELECT', 'INSERT', 'UPDATE', 'DELETE'],
    'audit_log': ['SELECT']
  },
  'editor': {
    'stud_info': ['SELECT', 'INSERT', 'UPDATE'],
    'users': ['SELECT'],
    'audit_log': []
  },
  'viewer': {
    'stud_info': ['SELECT'],
    'users': [],
    'audit_log': []
  }
}
```

3. **Secure Query Builder:**

```
def execute_query(user_role, table, operation, filters=None):
    # Check permissions
    # Validate and sanitize inputs
    # Use parameterized queries
    # Log the operation
```

Expected Output

=== DATABASE CONNECTION ===

User: editor1

Role: Editor

Connecting to MySQL database: stud_db

=== ATTEMPTING QUERY ===

Query: SELECT * FROM stud_info WHERE branch = 'CSE'

Permission check: ✓ Editor can SELECT from stud_info

Input validation: ✓ 'CSE' is valid string

Using prepared statement...

Query executed successfully

Results: 15 records returned

=== ATTEMPTING UNAUTHORIZED QUERY ===

User: viewer1 attempting: DELETE FROM stud_info WHERE roll='CS101'

Permission check: ✗ Viewer cannot DELETE from stud_info

Action: Query rejected, logged as security violation

=== ATTEMPTING SQL INJECTION ===

Input: ' OR '1'='1

Detection: ✗ Potential SQL injection detected

Action: Query blocked, user logged out

Audit log updated

=== QUERY LOG ===

```
+--------------------+---------+----------+--------------------------------+-----------+
| Timestamp          | User    | Table    | Query                          | Status    |
+--------------------+---------+----------+--------------------------------+-----------+
| 14:30:22           | editor1 | stud_info| SELECT * FROM stud_info        | ALLOWED   |
| 14:31:15           | viewer1 | stud_info| DELETE ...                     | DENIED    |
| 14:32:07           | attacker| stud_info| ' OR '1'='1                    | BLOCKED   |
+--------------------+---------+----------+--------------------------------+-----------+
```

---

8. JWT-based Access Control for Distributed Systems

Problem Statement

Implement a distributed authentication system using JSON Web Tokens with role-based access control, token refresh, and secure API endpoints.

Implementation Requirements

1. **JWT Token Structure:**

```
json
{
  "header": {
    "alg": "HS256",
    "typ": "JWT"
  },
  "payload": {
    "sub": "user123",
    "name": "Alice Johnson",
    "role": "editor",
    "iat": 1738531800,
    "exp": 1738535400,
    "jti": "a1b2c3d4e5"
  },
  "signature": "HMAC_SHA256(...)"
}
```

2. **Token Management:**
    - Access tokens (15 min expiry)
    - Refresh tokens (7 day expiry)
    - Token blacklisting for logout
    - Role-based endpoint protection

3. **API Endpoints:**

POST /api/auth/login

POST /api/auth/refresh

POST /api/auth/logout

GET /api/students (role: viewer, editor, admin)

POST /api/students (role: editor, admin)

PUT /api/students/:id (role: editor, admin)

DELETE /api/students/:id (role: admin)

GET /api/admin/users (role: admin)

Expected Output

=== LOGIN REQUEST ===

POST /api/auth/login

Body: {"username": "alice", "password": "********"}

Response:


```
{
  "status": "success",
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_in": 900,
  "user": {
    "id": "user123",
    "name": "Alice Johnson",
    "role": "editor"
  }
}
```

=== ACCESSING PROTECTED ENDPOINT ===

GET /api/students

Headers: Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Token validation:

✓ Signature valid

✓ Not expired (issued: 14:30, now: 14:35)

✓ Role "editor" has permission for GET /api/students

✓ Token not in blacklist

Response: 200 OK with student data

=== ACCESSING ADMIN ENDPOINT (Non-Admin) ===

GET /api/admin/users

Headers: Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Token validation:

✓ Signature valid

✓ Not expired

✗ Role "editor" cannot access admin endpoints

Response: 403 Forbidden


```
{
  "error": "Insufficient permissions",
  "required_role": "admin",
  "user_role": "editor"
}
```

=== TOKEN REFRESH ===

POST /api/auth/refresh

Body: {"refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."}

Refresh token validation:

✓ Token valid

✓ Not expired

✓ Not revoked

Response:

{

"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",

"expires_in": 900

}

=== TOKEN BLACKLISTING ===

POST /api/auth/logout

Headers: Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Adding token to blacklist...

Setting refresh token as revoked...

Response:

{

"status": "success",

"message": "Logged out successfully"

}

Subsequent request with same token: 401 Unauthorized

---

9. Cryptography for Big Data Security

Problem Statement

Design an encryption system for Hadoop HDFS that provides:

- Per-block encryption with unique keys
- Key management service
- Performance-optimized encryption/decryption
- Integration with Hadoop ecosystem

Implementation Requirements

1. **Encryption Architecture:**
    - Use AES-256-GCM for block encryption
    - Each HDFS block (128MB) gets unique data encryption key (DEK)
    - DEKs encrypted with master key (KEK)
    - Store metadata in separate secure store
2. **Components:**

```
// Encryption handler for HDFS
public class HDFSEncryptor {
  public EncryptedBlock encryptBlock(byte[] data, String blockId);
  public byte[] decryptBlock(EncryptedBlock encrypted, String blockId);
}

// Key Management Service
public class KMSClient {
  public String generateDEK(String blockId);
  public String encryptDEK(String dek, String kekId);
  public String decryptDEK(String encryptedDEK, String kekId);
}
```

3. **Performance Optimizations:**
    - AES-NI hardware acceleration
    - Parallel encryption of blocks
    - Caching of frequently used DEKs
    - Lazy decryption on read

Expected Output

=== BIG DATA ENCRYPTION SYSTEM ===

HDFS Cluster: 10 nodes, 1TB total

=== ENCRYPTING DATASET ===

Dataset: sales_data.parquet (500GB)

Block size: 128MB

Total blocks: 4000

Starting encryption:

[1/4000] Block: blk_1073741825

  Generating DEK: dek_7a4b9c...

  Encrypting with AES-256-GCM

  Encryption time: 45ms

  Overhead: 16 bytes (authentication tag)

[2/4000] Block: blk_1073741826

  ...

[4000/4000] Complete!

Encryption Summary:

Total data: 500GB

Encrypted size: 500GB + 62.5MB (metadata)

Throughput: 2.1 GB/s

Average block encryption: 42ms

=== KEY MANAGEMENT ===

Master Keys (KEK):

- kek_hdfs_2025_01 (active)

- kek_hdfs_2024_12 (previous)

Data Keys (DEK):

Total DEKs generated: 4000

Storage: Encrypted in KMS, metadata in HDFS

=== DECRYPTION ON READ ===

Reading file: /encrypted/sales_data.parquet

Request block: blk_1073741825

Decryption process:

1. Fetch encrypted block from HDFS

2. Retrieve encrypted DEK from metadata

3. Decrypt DEK using KEK: 5ms

4. Decrypt block data: 38ms

5. Verify authentication tag: ✓ Valid

Performance:

Read throughput: 1.8 GB/s

Decryption overhead: ~12%

=== KEY ROTATION ===

Rotating master key...

New KEK: kek_hdfs_2025_02

Re-encrypting DEKs: 4000 keys

Progress: 100%

Time: 2 minutes 15 seconds

---

10. Homomorphic Encryption System

Problem Statement

Implement a Paillier homomorphic encryption system that supports addition and multiplication (by constant) operations on encrypted data without decryption.

Implementation Requirements

1. **Paillier Cryptosystem Implementation:**

```
class Paillier:
  def generate_keys(key_size=2048):
    # Generate p, q primes
    # Compute n = p*q
    # Public key: (n, g)
    # Private key: (lambda, mu)

  def encrypt(public_key, m):
    # m in Z_n
    # Select random r
    # ciphertext = g^m * r^n mod n^2

  def decrypt(private_key, c):
    # m = L(c^lambda mod n^2) * mu mod n

  def add(public_key, c1, c2):
    # c3 = c1 * c2 mod n^2

  def multiply(public_key, c, k):
    # c' = c^k mod n^2
```

2. **Operations Support:**
    - Homomorphic addition: Enc(a) + Enc(b) = Enc(a + b mod n)
    - Homomorphic multiplication by constant: k * Enc(a) = Enc(k * a mod n)

Expected Output

=== HOMOMORPHIC ENCRYPTION DEMO ===

=== KEY GENERATION ===

Generating Paillier keys (2048-bit)...

Public key: (n, g)

n = 323170060713110073007148766886699519604441026697154840321303...

g = n + 1

Private key generated

Key generation time: 1.2 seconds

=== ENCRYPTION ===

Plaintext numbers:

a = 42

b = 17

c = 100

Encrypting a: E(42)

Ciphertext: 284962735897234598723459872345987234598723459872345987...

Encryption time: 45ms

Encrypting b: E(17)

Ciphertext: 498723459872345987234598723459872345987234598723459872...

Encryption time: 42ms

Encrypting c: E(100)

Ciphertext: 598723459872345987234598723459872345987234598723459872...

Encryption time: 44ms

=== HOMOMORPHIC ADDITION ===

Operation: E(42) ⊕ E(17) = E(42 + 17) = E(59)

Computing on ciphertexts:

c_sum = E(42) * E(17) mod n^2

Computation time: 12ms

Result ciphertext: 887234598723459872345987234598723459872345987234...

Verification by decryption:

Decrypt(c_sum) = 59 ✓

Expected: 42 + 17 = 59 ✓

=== HOMOMORPHIC MULTIPLICATION BY CONSTANT ===

Operation: 3 ⊗ E(17) = E(3 * 17) = E(51)

Computing on ciphertext:

c_prod = E(17)^3 mod n^2

Computation time: 15ms

Result ciphertext: 457234598723459872345987234598723459872345987234...

Verification by decryption:

Decrypt(c_prod) = 51 ✓

Expected: 3 * 17 = 51 ✓

=== COMPLEX EXPRESSION ===

Expression: (a + b) * 2 + c

Step 1: E(a + b) = E(42) ⊕ E(17) = E(59)

Step 2: E((a + b) * 2) = 2 ⊗ E(59) = E(118)

Step 3: E((a + b) * 2 + c) = E(118) ⊕ E(100) = E(218)

All operations performed on encrypted data!

Final decryption: 218 ✓

Verification: (42 + 17) * 2 + 100 = 218 ✓

=== PERFORMANCE METRICS ===

Encryption: 43ms average

Decryption: 85ms average

Homomorphic addition: 12ms

Homomorphic multiplication: 15ms

Ciphertext size: 4096 bits (512 bytes)

Security: 2048-bit equivalent to RSA-2048

---

11. Secure Multiparty Computation (Sum Protocol)

Problem Statement

Implement a secure multi-party computation system where N participants can compute the sum of their private numbers without revealing individual values.

Implementation Requirements

1. **Secret Sharing Protocol:**
    - Use additive secret sharing
    - Each participant splits their secret into N shares
    - Distribute shares to other participants
    - Each participant sums received shares
    - Reveal partial sums to compute final result
2. **Implementation Details:**

```
class SMPCProtocol:
  def __init__(self, participants):
    self.n = len(participants)
    self.participants = participants
    self.modulus = 2**32  # 32-bit modulus

  def share_secret(self, secret, participant_id):
    # Generate n-1 random shares
    # Last share = secret - sum(shares) mod modulus
    # Return shares dict {pid: share}

  def compute_partial_sum(self, received_shares):
    # Sum all received shares mod modulus
    # Return partial sum

  def reconstruct_result(self, partial_sums):
    # Sum all partial sums mod modulus
    # Return final result
```

Expected Output

=== SECURE MULTIPARTY COMPUTATION ===

Participants: 4 (Alice, Bob, Charlie, David)

Modulus: 2^32 (4294967296)

Private inputs:

Alice: 42

Bob: 17

Charlie: 100

David: 255

Expected sum: 42 + 17 + 100 + 255 = 414

=== PHASE 1: SECRET SHARING ===

Alice's secret: 42

Generating shares:

Share to Bob: 15

Share to Charlie: 23

Share to David: 11

Alice's own share: -7 (mod 2^32: 4294967289)

Bob's secret: 17

Generating shares:

Share to Alice: 8

Share to Charlie: 5

Share to David: 2

Bob's own share: 2

Charlie's secret: 100

Generating shares:

Share to Alice: 35

Share to Bob: 40

Share to David: 22

Charlie's own share: 3

David's secret: 255

Generating shares:

Share to Alice: 80

Share to Bob: 90

Share to Charlie: 70

David's own share: 15

=== PHASE 2: SHARE DISTRIBUTION ===

Alice receives: [8 from Bob, 35 from Charlie, 80 from David]

Bob receives: [15 from Alice, 40 from Charlie, 90 from David]

Charlie receives: [23 from Alice, 5 from Bob, 70 from David]

David receives: [11 from Alice, 2 from Bob, 22 from Charlie]

=== PHASE 3: PARTIAL SUM COMPUTATION ===

Alice computes partial sum:

Own share: 4294967289

From Bob: 8

From Charlie: 35

From David: 80

Partial sum: 4294967412 mod 2^32 = 116

Bob computes partial sum:

Own share: 2

From Alice: 15

From Charlie: 40

From David: 90

Partial sum: 147

Charlie computes partial sum:

Own share: 3

From Alice: 23

From Bob: 5

From David: 70

Partial sum: 101

David computes partial sum:

Own share: 15

From Alice: 11

From Bob: 2

From Charlie: 22

Partial sum: 50

=== PHASE 4: RESULT RECONSTRUCTION ===

Collecting partial sums:

Alice: 116

Bob: 147

Charlie: 101

David: 50

Final sum: 116 + 147 + 101 + 50 = 414 mod 2^32

=== VERIFICATION ===

Computed sum: 414

Expected sum: 414 ✓

12. Secure Data Access for Big Data Services API

Problem Statement

Create a secure REST API gateway for big data services with:

- JWT-based authentication
- Role-based access control
- Request/response encryption
- Comprehensive audit logging

Implementation Requirements

1. **API Gateway Components:**

API Gateway (Nginx + Lua/Go)

├── Authentication Middleware

├── Rate Limiting

├── Request Validation

├── Response Encryption

└── Audit Logger

2. **Security Headers:**

http

X-API-Key: encrypted_api_key

Authorization: Bearer <JWT>

X-Request-ID: uuid4

X-Timestamp: unix_epoch

X-Signature: HMAC_SHA256(body + timestamp + secret)

3. **Endpoint Examples:**

 # Spark Job Submission

POST /api/v1/spark/jobs

Body: {"job_config": encrypted, "data_source": "hdfs://..."}

 # Hive Query

POST /api/v1/hive/query

Body: {"query": encrypted_sql, "database": "sales"}

 # HDFS Access

GET /api/v1/hdfs/file/{path}

Query: ?offset=0&length=1024

Expected Output

=== SECURE BIG DATA API GATEWAY ===

Gateway URL: https://api.bigdata.company.com

Port: 443 (TLS 1.3 only)

=== AUTHENTICATION FLOW ===

Client request:

POST /api/v1/auth/login

Headers:

X-API-Key: ENC[AES256](client_id:timestamp)

Content-Type: application/json

Body: {"username": "data_scientist", "password": "********"}

Gateway processing:

1. Decrypt API key: ✓ Valid client

2. Verify credentials: ✓ Valid user

3. Check rate limit: ✓ Within limits (100 req/min)

4. Generate JWT: ✓ Issued with 15min expiry

5. Log audit entry: ✓ Recorded

Response:

HTTP/2 200 OK

Headers:

X-Request-ID: 7d9e2f1a-4b3c-8d5e-6f7a-1b2c3d4e5f6a

X-RateLimit-Limit: 100

X-RateLimit-Remaining: 99

X-RateLimit-Reset: 60

Body (encrypted):

{

"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",

```
  "token_type": "Bearer",
  "expires_in": 900,
  "permissions": ["spark:read", "hive:query", "hdfs:read"]
}
```

=== SPARK JOB SUBMISSION ===

Client request:

POST /api/v1/spark/jobs

Headers:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
X-Signature: sha256=4a8b9c2d7e1f3a5b...
X-Timestamp: 1738531800
```

Body (encrypted with session key):

```
{
  "job_name": "sales_analysis",
  "main_class": "com.company.SalesAnalyzer",
  "jars": ["hdfs:///jobs/sales-analyzer.jar"],
  "config": {"spark.executor.memory": "4g"},
  "data_source": "hdfs:///data/sales/2025/"
}
```

Gateway processing:

1. JWT validation: ✓ Valid, not expired
2. Signature verification: ✓ Valid
3. Permission check: ✓ User has "spark:submit"
4. Request decryption: ✓ Successful
5. Input validation: ✓ All parameters valid
6. Submit to Spark: ✓ Job ID: spark-202502031430-0001

Response:

HTTP/2 202 Accepted

Body:

```
 {
   "job_id": "spark-202502031430-0001",
   "status": "ACCEPTED",
   "tracking_url": "https://spark.company.com/ui/...",
   "estimated_completion": "2025-02-03T14:45:00Z"
 }
```

=== HDFS FILE ACCESS ===

Client request:

GET /api/v1/hdfs/file/data/sales/2025/01.csv

Query: ?offset=0&length=1048576 # First 1MB

Headers:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Gateway processing:

1. JWT validation: ✓ Valid
2. Path validation: ✓ Authorized path
3. Permission check: ✓ User has "hdfs:read"
4. File access check: ✓ File exists, user has permissions
5. Retrieve from HDFS: ✓ 1MB read in 45ms
6. Encrypt response: ✓ AES-256-GCM with fresh IV

Response:

HTTP/2 200 OK

Headers:

X-Encryption-Algorithm: AES-256-GCM

X-IV: 7a4b9c2d8e1f3a5b...

X-Auth-Tag: sha256=9d8e7f6a5b4c3d2e...

Body (encrypted):

<binary file data>

=== AUDIT LOG ENTRY ===


```
{
  "timestamp": "2025-02-03T14:30:22Z",
  "request_id": "7d9e2f1a-4b3c-8d5e-6f7a-1b2c3d4e5f6a",
  "user": "data_scientist",
  "ip": "203.0.113.45",
  "endpoint": "/api/v1/hdfs/file/data/sales/2025/01.csv",
  "method": "GET",
  "status_code": 200,
  "response_size": 1048576,
  "processing_time": 87,
  "permissions_used": ["hdfs:read"]
}
```


=== SECURITY METRICS ===

Requests processed: 1,245,789

Authentication failures: 127 (0.01%)

Rate limit violations: 89 (0.007%)

Permission denials: 432 (0.035%)

Average response time: 145ms

Encryption overhead: 8.2%
